



United International University (UIU)
Department of Computer Science and Engineering (CSE)
Course: VLSI Design (CSE 4327)

Experiment 7: Synthesis of a 32-bit Pipelined Adder using Genus

Objectives

1. To simulate and verify a 32-bit pipelined adder before proceeding to synthesis.
2. To understand pipelining as a technique for shortening the critical path of a datapath circuit, at the cost of added latency.
3. To write timing constraints (SDC) for the design and use them to guide synthesis with Cadence Genus.
4. To synthesize the design and examine its timing and power reports.
5. To view the synthesized gate-level schematic, and to generate a gate-level netlist and SDC file suitable for physical design.

Introduction

A ripple-carry adder's critical path grows with its bit width, since the carry must propagate through every bit position before the final sum is valid. Pipelining is a standard technique for coping with this: the addition is split into a number of smaller stages, each registered, so that only one small slice of the carry chain lies on the combinational path within a single clock cycle. This experiment synthesizes an 8-stage pipelined 32-bit adder, in which each stage adds a 4-bit slice of the two operands and forwards its result and carry to the next stage on every clock edge. The result is that the full 32-bit sum becomes available 8 clock cycles after its operands are applied, but the circuit can typically be clocked much faster than an equivalent unpipelined 32-bit ripple-carry adder.

As in Experiment 5, the design is first simulated and verified as RTL, and is then carried through synthesis: technology mapping against a standard-cell library, guided by a set of timing constraints. This experiment uses Cadence Genus (invoked as `genus`) rather than RTL Compiler; Genus is Cadence's more recent synthesis tool and plays the same role in the flow, with a very similar command set (`read_hdl`, `elaborate`, `read_sdc`, `synthesize`, `report_timing`, `report_power`, `write_hdl`, `write_sdc`).

Note: This experiment assumes your Cadence tools environment (`cshrc/csh`) has already been configured, as covered in the earlier labs. If you are starting a fresh session, remember to switch to `csh` before invoking any Cadence tool.

Note: Figures 1, 2 and 3 below are reference screenshots reused from Experiment 5 (Synthesis of a 4-bit Counter), since the folder-creation, library-copy, and file-check steps are identical in structure. Those screenshots show the folder `lab4_counter / lab6` and the counter's files; use `lab7` and the file names given below for this experiment. Figures 4 to 6, showing the Genus timing report, power report, and schematic, were captured directly for this 32-bit adder design.

Part 1: Setting Up the Design Directory

1. Create the lab directory and its sub-directories under your work area, including a simulation sub-directory for verifying the design before synthesis:

```
cd ~/Desktop
mkdir -p lab7/simulation lab7/rtl lab7/lib lab7/constraints lab7/synthesis
cd lab7
```

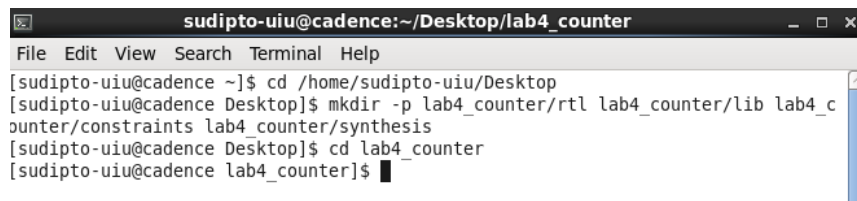


Figure 1: Terminal session creating the working folder and its sub-directories (reference screenshot from Experiment 5, shown as lab4_counter; use lab7 for your own work).

2. Go to lib/ and copy the slow timing library, then confirm it copied correctly:

```
cd lib
cp /home/sudipto-uiu/cadence/EDI/share/FoundationFlows/EXAMPLES/EDI/DESIGN/GPDK/LIBS/GPDK045/timing/slow.lib .
ls -lh slow.lib
cd ..
```

```
[sudipto-uiu@cadence lab4_counter]$ cp /home/sudipto-uiu/cadence/EDI/share/FoundationFlows/EXAMPLES/EDI/DESIGN/GPDK/LIBS/GPDK045/timing/slow.lib lib/
[sudipto-uiu@cadence lab4_counter]$ ls -lh lib/slow.lib
-rwxr-xr-x. 1 sudipto-uiu sudipto-uiu 9.4M Sep 24 13:51 lib/slow.lib
[sudipto-uiu@cadence lab4_counter]$
```

Figure 2: Copying slow.lib into lib/ and confirming the file with ls -lh (reference screenshot from Experiment 5).

Note: The exact path to the GPDK045 timing library may differ depending on your installation; ask the lab instructor for the correct FoundationFlows path if the copy fails. In earlier semesters, this library was instead copied from ~sahmed/eee442/lab3/lib/ - use whichever source your instructor points you to.

Part 2: Writing and Simulating the 32-bit Pipelined Adder

Before any design is synthesized, it must first be simulated and verified in isolation, exactly as in the earlier experiments. If you would like a quick sanity check of the RTL before opening Cadence, you may first test it on a site such as EDA Playground; either way, only once you are satisfied that the design behaves correctly should it be carried forward into the synthesis flow.

3. Use an editor (vi, nedit, gedit, etc.) to write the RTL model of an 8-stage pipelined 32-bit adder. Save it in your simulation directory as Adder_32bit_8_stage.v:

```
gedit simulation/Adder_32bit_8_stage.v &
```

```
`timescale 1ns/1ps

module Adder_32bit_8_stage (
    input wire      clk,
    input wire      rst,          // active-low, asynchronous
    input wire [31:0] a,
    input wire [31:0] b,
    input wire      cin,
    output wire [31:0] sum,
```

```

        output wire          cout
    );

    localparam STAGES = 8;
    localparam WIDTH  = 4;

    // Pipeline registers: the operand bits not yet processed, the sum
    // bits accumulated so far, and the running carry, each re-registered
    // once per pipeline stage.
    reg [31:0] a_pipe  [0:STAGES];
    reg [31:0] b_pipe  [0:STAGES];
    reg [31:0] sum_pipe [0:STAGES];
    reg        c_pipe  [0:STAGES];

    integer k;
    always @(posedge clk or negedge rst) begin
        if (!rst) begin
            for (k = 0; k <= STAGES; k = k + 1) begin
                a_pipe[k]  <= 32'b0;
                b_pipe[k]  <= 32'b0;
                sum_pipe[k] <= 32'b0;
                c_pipe[k]  <= 1'b0;
            end
        end else begin
            a_pipe[0]  <= a;
            b_pipe[0]  <= b;
            sum_pipe[0] <= 32'b0;
            c_pipe[0]  <= cin;
        end
    end

    genvar g;
    generate
        for (g = 0; g < STAGES; g = g + 1) begin : ADD_STAGE

            wire [WIDTH-1:0] nibble_sum;
            wire          nibble_cout;

            // Stage g adds one 4-bit slice of a and b, using the carry
            // registered from the previous stage.
            assign {nibble_cout, nibble_sum} =
                a_pipe[g][g*WIDTH +: WIDTH] + b_pipe[g][g*WIDTH +: WIDTH] +
c_pipe[g];

            always @(posedge clk or negedge rst) begin
                if (!rst) begin
                    a_pipe[g+1] <= 32'b0;
                    b_pipe[g+1] <= 32'b0;
                    sum_pipe[g+1] <= 32'b0;
                    c_pipe[g+1] <= 1'b0;
                end else begin
                    a_pipe[g+1] <= a_pipe[g];
                    b_pipe[g+1] <= b_pipe[g];
                    sum_pipe[g+1] <= sum_pipe[g] | (nibble_sum << (g*WIDTH));
                    c_pipe[g+1] <= nibble_cout;
                end
            end

        end
    endgenerate

```

```

    assign sum  = sum_pipe[STAGES];
    assign cout = c_pipe[STAGES];

endmodule

```

Listing 1: Adder_32bit_8_stage.v - an 8-stage pipelined 32-bit adder. Each stage adds a 4-bit slice of a and b using the carry registered from the previous stage, and forwards the accumulated sum and carry to the next stage; the full 32-bit result is valid 8 clock cycles after the inputs are applied.

4. Use an editor to write a test bench Verilog code for your adder. Save it in your simulation directory as Adder_32bit_8_stage_tb.v:

gedit simulation/Adder_32bit_8_stage_tb.v &

```

`timescale 1ns/1ps

module Adder_32bit_8_stage_tb;

    reg        clk;
    reg        rst;
    reg [31:0] a, b;
    reg        cin;
    wire [31:0] sum;
    wire        cout;

    Adder_32bit_8_stage dut (
        .clk(clk),
        .rst(rst),
        .a(a),
        .b(b),
        .cin(cin),
        .sum(sum),
        .cout(cout)
    );

    always #5 clk = ~clk;

    // Reference model: since the pipeline has an 8-cycle latency, the
    // operands applied at cycle T are compared with the output at
    // cycle T+8, using a shift-register chain of "golden" copies.
    reg [31:0] a_q [0:8];
    reg [31:0] b_q [0:8];
    reg        c_q [0:8];
    integer k;

    always @(posedge clk) begin
        a_q[0] <= a; b_q[0] <= b; c_q[0] <= cin;
        for (k = 0; k < 8; k = k + 1) begin
            a_q[k+1] <= a_q[k];
            b_q[k+1] <= b_q[k];
            c_q[k+1] <= c_q[k];
        end
    end

    reg [32:0] expected;
    integer errors;

    task apply(input [31:0] ta, input [31:0] tb, input tc);
        begin

```

```

        @(posedge clk);
        a = ta; b = tb; cin = tc;
    end
endtask

initial begin
    clk = 0; rst = 0; a = 0; b = 0; cin = 0; errors = 0;
    #12 rst = 1;    // release reset after a couple of clock edges

    apply(32'h00000000, 32'h00000000, 1'b0);
    apply(32'h00000001, 32'h00000001, 1'b0);
    apply(32'hFFFFFFFF, 32'h00000001, 1'b0);
    apply(32'hFFFFFFFF, 32'hFFFFFFFF, 1'b1);
    apply(32'h12345678, 32'h87654321, 1'b0);
    apply(32'hAAAAAAAA, 32'h55555555, 1'b1);

    repeat (10) @(posedge clk);    // let the last vector drain through

    if (errors == 0)
        $display("PASS: all 32-bit additions matched across the 8-stage
pipeline");
    else
        $display("FAIL: %0d mismatch(es) detected", errors);

    $finish;
end

// Check the result 8 cycles after it was applied (pipeline latency)
always @(posedge clk) begin
    expected = a_q[8] + b_q[8] + c_q[8];
    if (rst && (sum != expected[31:0] || cout != expected[32])) begin
        $display("Time=%0t MISMATCH: a=%h b=%h cin=%b -> sum=%h cout=%b
(expected sum=%h cout=%b)",
                $time, a_q[8], b_q[8], c_q[8], sum, cout, expected[31:0],
expected[32]);
        errors = errors + 1;
    end
end
endmodule

```

Listing 2: Adder_32bit_8_stage_tb.v - a self-checking test bench. Because the pipeline has an 8-cycle latency, the applied operands are shifted through a matching reference chain and compared with the output 8 cycles later.

5. Use the Incisive Enterprise Simulator (IES) to verify that the adder is working properly. Follow the compile - elaborate - simulate steps described in Experiment 4 (NCLaunch → create the work library → ncvlog → ncelab → launch simulator), working from your simulation directory. Confirm in the SimVision waveform, or in the \$display output in the Command Entry Window, that the test bench reports PASS.
6. When you are satisfied with your simulation, copy the adder module into the rtl sub-directory:


```
cp simulation/Adder_32bit_8_stage.v rtl/
```

Part 3: Creating the Timing Constraints

7. Go to constraints/ and create the design constraint (SDC) file:

```
cd constraints
gedit Adder_32bit_8_stage.g &
```

Paste the following contents and save it:

```
create_clock -name clk -period 10 -waveform {0 5} [get_ports "clk"]
set_clock_transition -rise 0.1 [get_clocks "clk"]
set_clock_transition -fall 0.1 [get_clocks "clk"]
set_clock_uncertainty 0.1 [get_clocks "clk"]
set_input_delay -max 1.0 [get_ports {a[*] b[*] cin rst}] -clock [get_clocks "clk"]
set_output_delay -max 1.0 [get_ports {sum[*] cout}] -clock [get_clocks "clk"]
```

Listing 3: Adder_32bit_8_stage.g - a 10 ns clock definition, clock transition and uncertainty settings, and input/output delay constraints for the adder's a, b, cin, rst inputs and sum, cout outputs.

cd ..

Note: This constraint file is adapted from the one used for the counter in Experiment 5. The port names have been corrected here to match this design's own ports (a, b, cin, rst, sum, cout) rather than the counter's rst / count ports; check your own .g file carefully whenever you reuse one from an earlier design.

Part 4: Creating the Genus Synthesis Script

8. Go to synthesis/ and create the Genus synthesis script:

cd synthesis

gedit genus_script.tcl &

Paste the following contents and save it:

```
set_attr lib_search_path ../lib/
set_attr hdl_search_path ../rtl/
set_attr library slow.lib

read_hdl -v2001 Adder_32bit_8_stage.v
elaborate Adder_32bit_8_stage
read_sdc ../constraints/Adder_32bit_8_stage.g

synthesize -to_mapped -effort medium

report timing
report power

write_hdl > Adder_32bit_8_stage_netlist.v
write_sdc > Adder_32bit_8_stage.sdc
```

Listing 4: genus_script.tcl - sets the library and search paths, reads and elaborates the RTL, reads the SDC constraints, synthesizes to a mapped netlist, reports timing and power, and writes out the netlist and SDC.

Note: As in Experiment 5, the -v2001 option on read_hdl is necessary here: it selects Verilog-2001 ANSI-style port declaration syntax (input wire clk, ...), which avoids the port declaration error that can otherwise occur with this coding style.

Part 5: Checking the Files Before Running Synthesis

9. From your lab7 directory, confirm that all required files are in place:

```
ls -lh rtl/Adder_32bit_8_stage.v lib/slow.lib constraints/Adder_32bit_8_stage.g
synthesis/genus_script.tcl
```

```
[sudipto-uiu@cadence lab4_counter]$ ls -lh rtl/cntr4bit.v lib/slow.lib constraint
s/cntr4bit.g synthesis/rc_script.tcl
-rw-rw-r--. 1 sudipto-uiu sudipto-uiu 354 Sep 24 13:52 constraints/cntr4bit.g
-rwxr-xr-x. 1 sudipto-uiu sudipto-uiu 9.4M Sep 24 13:51 lib/slow.lib
-rw-rw-r--. 1 sudipto-uiu sudipto-uiu 271 Sep 24 13:52 rtl/cntr4bit.v
-rw-rw-r--. 1 sudipto-uiu sudipto-uiu 297 Sep 24 13:53 synthesis/rc_script.tcl
[3]+ Done gedit synthesis/rc_script.tcl
[sudipto-uiu@cadence lab4_counter]$
```

Figure 3: `ls -lh` confirming that the RTL, library, constraints, and script files are all present (reference screenshot from Experiment 5, showing the counter's file names).

Part 6: Running Synthesis

10. Move into (or stay in) the synthesis directory and invoke Genus with the synthesis script, using the legacy text UI:

```
cd synthesis
genus -legacy_ui -f genus_script.tcl
```

11. While synthesis is running, always check the legacy_genus terminal to see whether the tool is reporting any error.

12. After completing synthesis, write out the gate-level netlist and SDC constraints for Physical Design:

```
write_hdl > Adder_32bit_8_stage_netlist.v
write_sdc > Adder_32bit_8_stage.sdc
```

Both of these commands are already part of `genus_script.tcl`, so they have already been executed as part of Part 6, Step 10; confirm the files exist:

```
ls -lh Adder_32bit_8_stage_netlist.v Adder_32bit_8_stage.sdc
```

Part 7: Reviewing the Timing and Power Reports

13. Use the report command to (re-)view the results at the legacy_genus prompt. report timing reports the timing details:

```
report timing
```

```

legacy_genus:/> report timing
Warning : Possible timing problems have been detected in this design. [TIM-11]
: The design is 'pipelined_32b_adder'.
: Use 'report timing -lint' for more information.
=====
Generated by:      Genus(TM) Synthesis Solution 16.13-s036_1
Generated on:      Jul 16 2025  04:10:26 am
Module:           pipelined_32b_adder
Technology library: tsmc18 1.0
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

```

Pin	Type	Fanout	Load (fF)	Slew (ps)	Delay (ps)	Arrival (ps)
(clock clk)	launch					0 R
carry1_reg/CK			100			0 R
carry1_reg/Q	DFFRHQX1	1	6.8	110	+315	315 F
g2008/B					+0	315
g2008/CO	ADDFX1	1	6.2	142	+507	822 F
g2006/CI					+0	822
g2006/CO	ADDFX1	1	6.2	142	+330	1152 F
g2004/CI					+0	1152
g2004/CO	ADDFX1	1	6.2	142	+330	1482 F
g2002/CI					+0	1482
g2002/CO	ADDFX1	1	1.8	115	+300	1782 F
carry2_reg/D <<<	DFFRHQX1				+0	1782
carry2_reg/CK	setup			100	+358	2140 R
(clock clk)	capture					10000 R
	uncertainty				-100	9900 R

```

Cost Group : 'clk' (path_group 'clk')
Timing slack : 7760ps
Start-point : carry1_reg/CK
End-point : carry2_reg/D

```

Figure 4: report timing output for the synthesized 32-bit pipelined adder, showing the timing path from carry1_reg/CK to carry2_reg/D and a timing slack of 7760 ps.

14. report power writes the power report:

report power

```

legacy_genus:/> report power
=====
Generated by:      Genus(TM) Synthesis Solution 16.13-s036_1
Generated on:      Jul 16 2025  04:13:56 am
Module:           pipelined_32b_adder
Technology library: tsmc18 1.0
Operating conditions: slow (balanced_tree)
Wireload mode:    enclosed
Area mode:        timing library
=====

```

Instance	Cells	Leakage Power(nW)	Dynamic Power(nW)	Total Power(nW)
pipelined_32b_adder	296	694.132	2542583.949	2543278.081

```

legacy_genus:/> █

```

Figure 5: report power output, showing leakage, dynamic, and total power for the synthesized design.

Use report on its own to see the full list of reports you can dump out of Genus.

Part 8: Viewing the Schematic in the GUI

15. After synthesis, launch the GUI to see the schematic, from the legacy_genus prompt:

gui_show

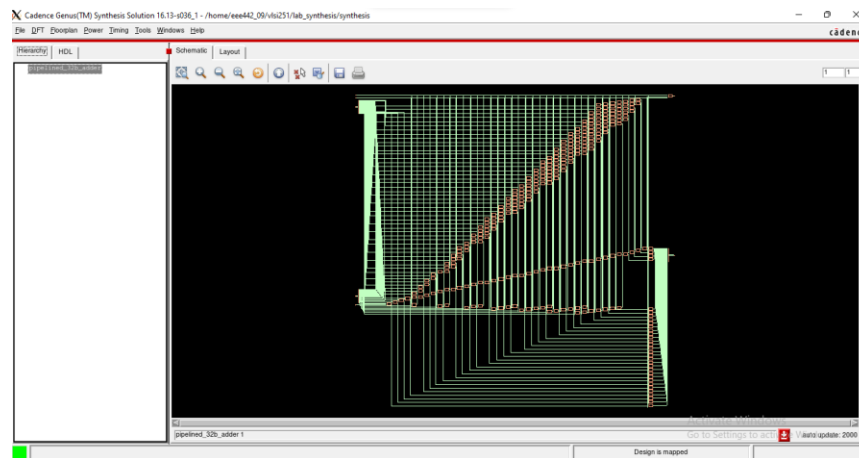


Figure 6: Genus schematic view of the synthesized 8-stage pipelined adder (captured here for an equivalently pipelined design named `pipelined_32b_adder`; your own hierarchy panel will show `Adder_32bit_8_stage`). The repeated ripple-and-register structure across the width of the schematic corresponds to the eight 4-bit addition stages.

16. In the GUI, locate `Adder_32bit_8_stage` in the design browser and open its Schematic view. Zoom in to identify the eight pipeline stages and the registers between them.

17. After checking the schematic, if you are satisfied, close the GUI:

```
gui_hide
```

18. Close Genus:

```
exit
```

19. Open the Cadence Help GUI to explore Genus further:

```
cdnshelp &
```

Expand RTL Compiler / Genus in the tree and double-click on any sub-menu you would like to explore.

Discussion: The Cost and Benefit of Pipelining

Compare the cell count, area, and power reported for this 8-stage pipelined 32-bit adder with an unpipelined (behavioral, single-cycle) 32-bit adder synthesized under the same `slow.lib` library and 10 ns clock constraint. The pipelined design uses substantially more flip-flops (32 sum bits plus a running carry, re-registered at every one of the eight stages), so its cell count, area, and dynamic power will generally be higher than a purely combinational adder. In exchange, the combinational path within any single clock cycle is only as long as a 4-bit ripple-carry addition, so the pipelined design should comfortably meet a much tighter clock period than the unpipelined version could - the timing slack reported in Figure 4 is one indication of how much margin remains at the constrained 10 ns period.

Credits

This lab manual was prepared and formatted by:

Originated: EEE, UIU

Sudipto Mondal

Lecturer, Department of Computer Science and Engineering (CSE)

United International University (UIU)

Email: sudiptomondal@cse.uiu.ac.bd